UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

Práctica No. 3

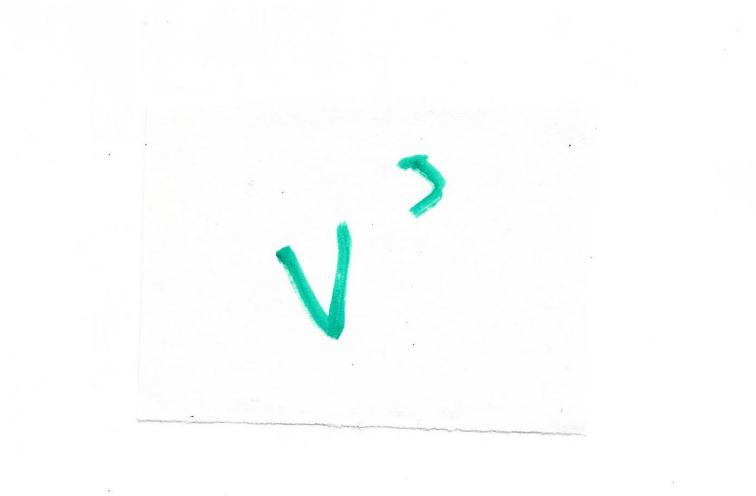
Compiladores

Estudiante

Nombre	Número de Cuenta
Fuentes Ortega Diego Tonatiuh	320067660
Carballido Camacateco Ana Lilia	315314601

13 de marzo de 2026

Validación



Vídeo de validación: Flex

Descripción de implementación

Estructura del analizador léxico

El analizador léxico fue implementado utilizando Lex, el cual permite definir patrones regulares que asocian acciones de C a cada uno de ellos. El analizador se encarga de leer el programa fuente caracter ppor caracter y lo divide por tokens.
El archivo se divide en tres secciones principales: definiciones, reglas léxicas y código auxiliar.

Definición

Se utiliza la opción:

```
%option noyywrap yylineno
```

Donde `noyywrap` se utiliza para indicar al anlizador que no lea archivos extra cuando se llega al final del arcvhivo de entrada. Luego, `yylineno` sirve como variable global que lleva un conteo de líneas procesadas.
El archivo `scanner.h` contiene las definiciones de los tokens que utiliza el analizador léxico.
Al final, se define un estado léxico llamado `COMMENT` que permite el manejo de comentarios multilínea.

Patrones léxicos

Se definiceron los siguientes macros para representar patrones del lenguaje:

- **Identificadores:** Comeinzan con una letra o guión bajo, luego pueden seguir letras, números o guiones bajos.
$$ID[a-zA-Z_][a-zA-Z_0-9]^*$$
- **Números enteros:** Se forman de al menos una cifra decimal.
$$INT[0-9]+$$
- **Números flotantes:** Permiten números con parte decimal y notación científica.
$$FLOAT((INT.INT)|(INT.INT)([eE][+-]?INT)|INT(e[+-]?INT))$$

- **Espacios en blanco:** Los espacios, tabuladores y saltos de línea son ignorados.

$$WS[trn]+$$

Comentarios

El analizador léxico reconoce dos tipos de comentarios, de una sola línea comenzando con el caracter `//` y multilínea que `(/*...*/)`. Para el manejo de estos comentarios, se definió el estado léxico `COMMENT`, en el cual todos los caracteres son ignorados hasta encontrar los símbolos de cierre, en ese caso el analizador vuelve al estado `INITIAL`. El analizador devuelve error si el fin del archivo se encuentra antes de terminar un comentario.

Reconocimiento de palabras reservadas

El analizador léxico reconoce las siguientes palabras reservadas:

- int
- float
- double
- char
- void
- if
- else
- while
- for
- return
- break
- continue

Cada una de las palabras reservadas genera un token correspondiente TOK_KW_*, donde * se reemplaza por la palabra reservada.

Operadores

El analizador distingue entre los siguientes operadores simples y compuestos:

- incremento y decremento $(++, --)$
- asignaciones compuestas $(+=, -=, *=, /=, \%=)$
- operadores relacionales $(==, !=, <=, >=)$
- operadores lógicos `(&&, ||)`

Los operadores compuestos se definieron antes de los operadores simples con el fin de evitar ambugüedades. Operadores simples:

- asignación $(=)$
- comparaciones $(<, >)$
- negación $(!)$
- operadores aritméticos $(+, -, *, /, \%)$

Delimitadores

Se definen los siguientes delimitadores para el lenguaje:

- paréntesis `(,)`
- llaves `,`
- corchetes `[]`
- coma `,`
- punto y coma `;`

Literales e identificadores

Los literales de caracter y cadena, admiten secuencias de escape, luego el analizador devuelve el caracter correspondiente.
Una palabra genera el token $TOK_IDENTIFIER$ cuando no coincide con ninguna palabra reservada, por lo cual, estas últimas se definieron primero.

Manejo de errores

Cuando el analizador alcanza el final del archivo, devuelve el token `TOK_EOF`.

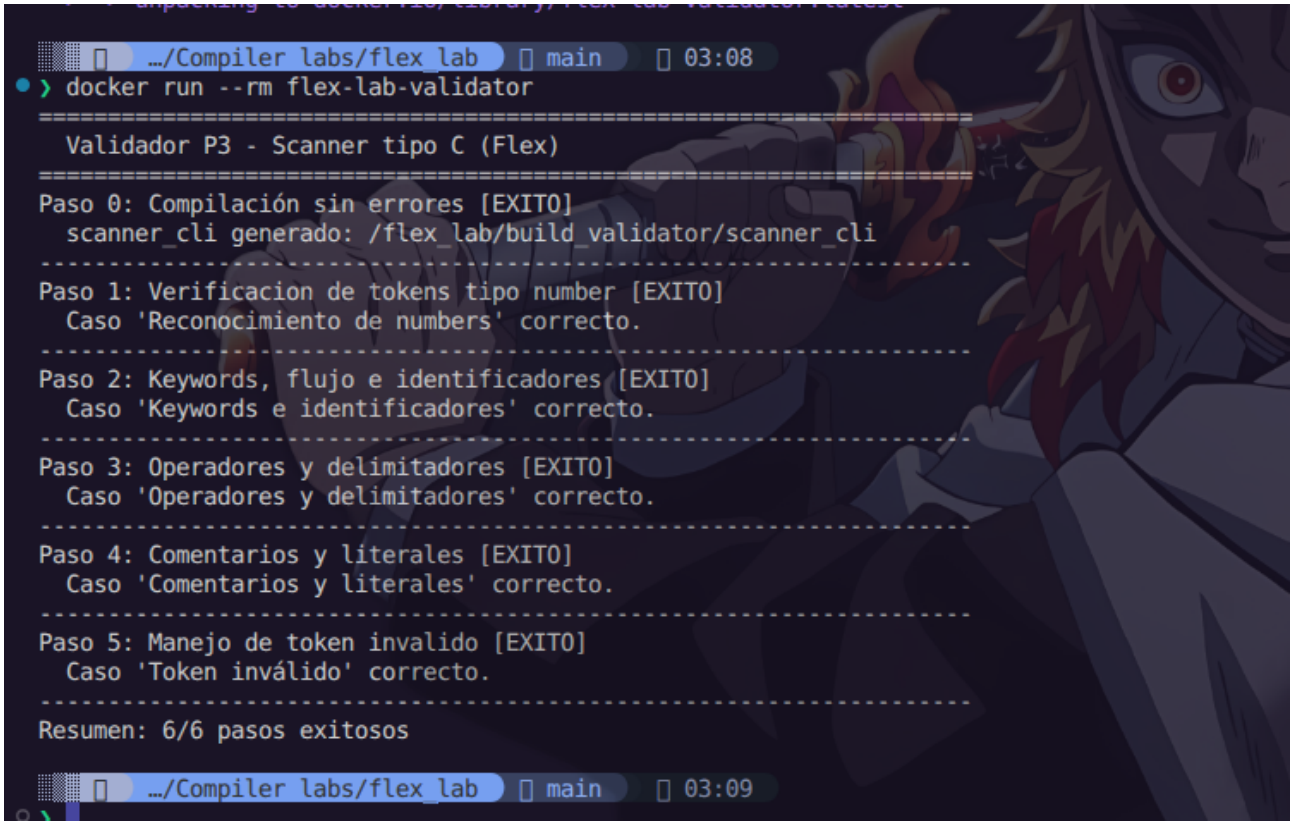
Decisiones de diseño

Debido a la jerarquía que maneja Flex cuando dos expresiones pueden coincidir con más de una regla, se decidió definir las reglas en el siguiente orden con el fin de evitar ambigüedades:

- Se definieron las palabras reservadas antes que los identificadores.
- Se definen los operadores compuestos antes de los operadores simples.
- Primero se reconocen literales fltantes y después las literales enteras.

Dificultades encontradas

Hubo una pequeña dificultad para corregir el archivo CMake ya que tenía otra estructura de en dónde estaba localizado el archivo scanner.c dentro de los directorios ya que en nuestro caso específico estaba dentro de un directorio src. Cabe mencionar que, aunque simple, sí me tomó un tiempo ver que ese era el error y no lo demás.

Otra dificultad encontrada fue la determinación del orden de las reglas léxicas con el fin de evitar ambugüedades. Para esto se utilizó la jerarquía que maneja Flexncon la cual se decide entre las reglas eligiendo la que se definió primero cuando una expresión coincide con más de una.

Resultados obtenidos